PR #218: docs(getting-started): add an Operating Tenstorrent Software overview and reorder Get Started

https://github.com/tenstorrent/tenstorrent.github.io/pull/218
Branch `software-stack-tt-cli` → `deploy-models-tt-cli` · author ndalyTT · head `b2d10129`

Rendered review copy of the pages this PR changes, built with Sphinx from the PR head (commit `b2d1012`; a stacked PR already contains its base branch), generated 2026-11-04. Pages render in the site theme with the navigation removed; code blocks wrap instead of scrolling.

Commits (20)

`41ac9953`	2026-11-04	docs(getting-started): add an Operating Tenstorrent Software overview
`c7376949`	2026-11-04	docs(getting-started): reorder Get Started, rename the overview, and …
`b2624556`	2026-11-04	docs(getting-started): plain-text headings on the operating page
`5ed810f9`	2026-11-04	docs(getting-started): rename the operating diagram file
`4558899c`	2026-11-04	docs(getting-started): label the left-hand arrows in the operating di…
`f60d7688`	2026-11-04	docs(getting-started): taller operating diagram with larger tiles
`32893eaa`	2026-11-04	docs(getting-started): size the Hugging Face Hub label like the infer…
`bae0b13e`	2026-11-04	docs(getting-started): show HugePages instead of Docker in the operat…
`30dccadb`	2026-11-04	docs(getting-started): short sidebar labels for the first three Get S…
`af1ea1cc`	2026-11-04	docs(getting-started): drop the Deploy LLMs card and link after the p…
`8a4f0a98`	2026-11-04	docs(getting-started): list the tt CLI first under Running Models on …
`c4a79d97`	2026-11-04	docs(getting-started): shorten the tt CLI bullet on the Building page
`2bb04881`	2026-11-04	docs(getting-started): say which path serves catalog, Studio-only, an…
`fb29e1a5`	2026-11-04	docs(getting-started): link tt-model-manager from the Building page
`1257a1c3`	2026-11-04	docs(getting-started): point the Building page intro at the Operating…
`6ff5450c`	2026-11-04	docs(getting-started): reword the tt paragraph on the Operating page
`0772d4fc`	2026-11-04	docs(getting-started): rewrite What tt Manages on the Operating page
`39434f0b`	2026-11-04	docs(getting-started): say why tt operating the layers matters
`2de4037c`	2026-11-04	docs(getting-started): drop the What tt Manages intro sentence
`b2d10129`	2026-11-04	docs(getting-started): name the Wormhole mesh step that tt update lea…

Files changed (8)

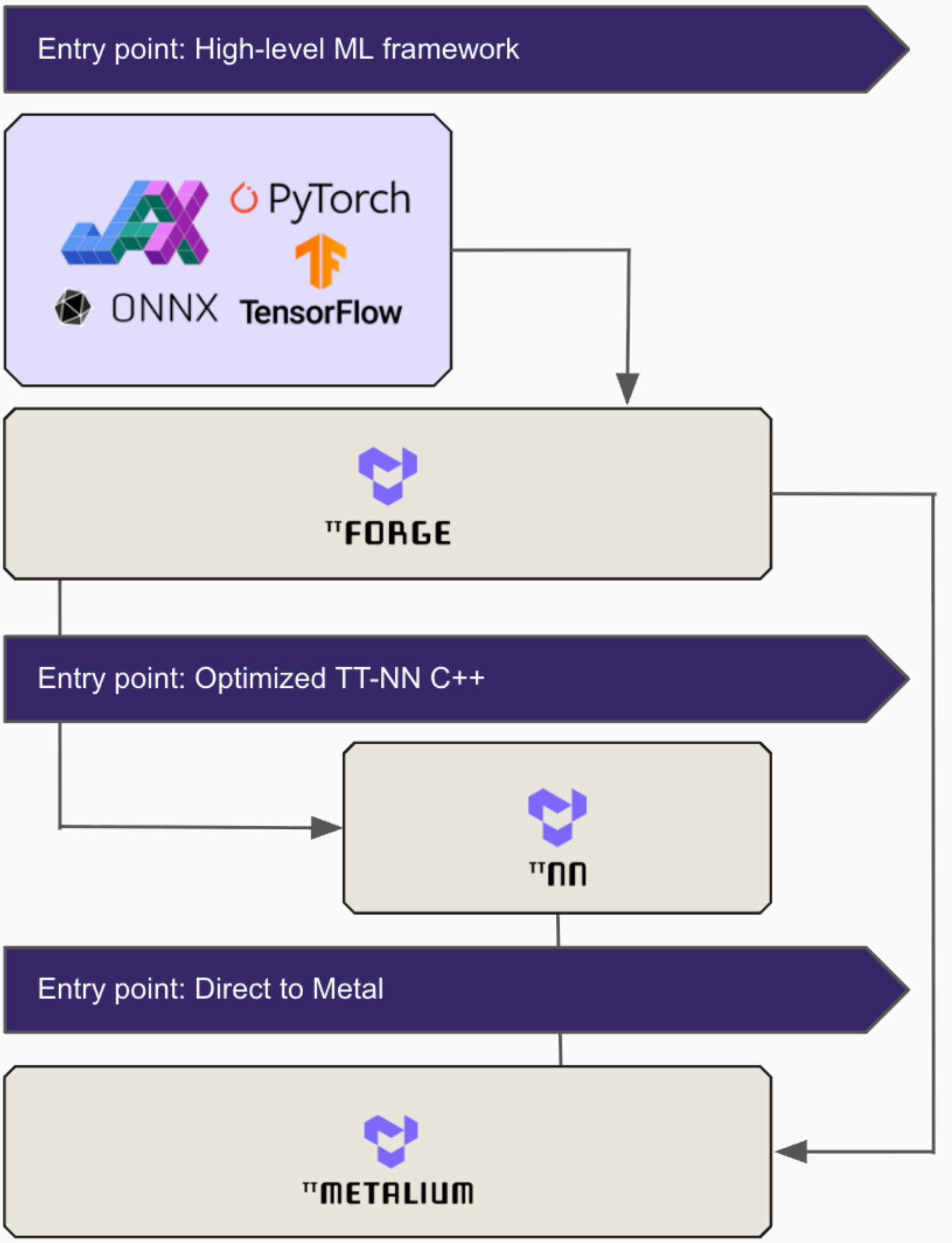
core/_extra/AGENTS.md	+1	-0
core/_extra/llms.txt	+3	-0
core/_templates/home.html	+19	-12
core/getting-started/README.md	+1	-1
core/getting-started/operating-tt-software-flow.svg	+65	-0
core/getting-started/operating-tt-software.md	+141	-0
core/getting-started/tt-software-stack.md	+6	-5
core/index.rst	+4	-3

Pages in this PDF

1. getting-started/tt-software-stack.html
2. getting-started/operating-tt-software.html
3. index.html
4. getting-started/README.html

Building with Tenstorrent Software

This document provides developers and researchers with an overview of the main components of the Tenstorrent software stack: TT-Metalium™, TT-NN™, and TT-Forge™. You will learn about each tool's purpose, use cases, and where to find its source code. If you are looking for how to operate your Tenstorrent hardware instead, such as installing the system software and serving models, see Operating Tenstorrent Software.


Entry point: High-level ML framework
PyTorch
ONNX
TensorFlow
TTFORGE
Entry point: Optimized TT-NN C++
TTNN
Entry point: Direct to Metal
TTMETALIUM

TT-Forge™: The MLIR-Based Compiler

TT-Forge™ is Tenstorrent's Multi-Level Intermediate Representation (MLIR)-based compiler. It bridges high-level machine learning frameworks with the Tenstorrent software stack.

Interact with TT-Forge™ to compile models from frameworks such as PyTorch, JAX, and TensorFlow for execution on Tenstorrent hardware. It offers an automated, general path to run many types of model architectures without requiring custom kernel development. TT-Forge™ integrates with and lowers to TT-Metalium for hardware execution.

The main project is located in the tt-forge GitHub repository.

TT-NN™: A Python & C++ Neural Network OP library

TT-NN™ is a library of neural network operations that provides a user-friendly interface for running models on Tenstorrent hardware. It is designed to be intuitive for developers familiar with PyTorch.

Interact with TT-NN™ to run AI models using a familiar, high-level Python API without managing the complexities of the underlying hardware. TT-NN™ builds upon TT-Metalium™ and provides a stable set of pre-packaged, optimized operations. It is also available with a C++ API.

The TT-NN™ library is part of the tt-metal GitHub repository.

TT-Metalium™: Programming Tenstorrent Hardware

TT-Metalium™ is the low-level, open-source software development kit (SDK) that provides developers direct access to Tenstorrent hardware. It is a bare-metal programming environment designed for users who must write custom C++ kernels for machine learning or other high-performance computing workloads.

Interact with TT-Metalium™ when you require complete control over the hardware to optimize code for performance, explicitly manage memory, or implement novel operations not found in standard libraries. This environment exposes the RISC-V processors, the Network-on-Chip (NoC), and the matrix and vector engines within each Tensix core.

The main project is available in the tt-metal GitHub repository.

Running Models on Hardware

Once you have the software stack installed, these are the primary entry points for running models:

- **Tenstorrent CLI** — `tt serve <model>` serves any model validated for your hardware as an OpenAI-compatible endpoint, running TT-Inference-Server for catalog models and tt-model-manager for community bundles, and handling the container, weights, and configuration.
- **TT-Inference-Server** — manages Docker containers, model downloads, and serving configuration, and provides an OpenAI-compatible API endpoint. It is the authoritative source for which models are validated on each hardware generation.
- **TT-Forge Models** — the catalog of models that have been validated against TT-Forge™. The starting point when compiling a PyTorch, ONNX, or JAX model against the stack.
- **TT-Studio** — a web interface that serves the same catalog for users who prefer a point-and-click model deployment flow.

For monitoring your hardware while models run, TT-Toplike reads power, temperature, and core activity in real time from the chips.

Exploring our Developer Hub

To continue learning, visit and explore our Developer Hub. There you will find in-depth articles, and information on our software bounty program. You can also find tutorials and technical overviews on our YouTube channel.

Engaging with our Community

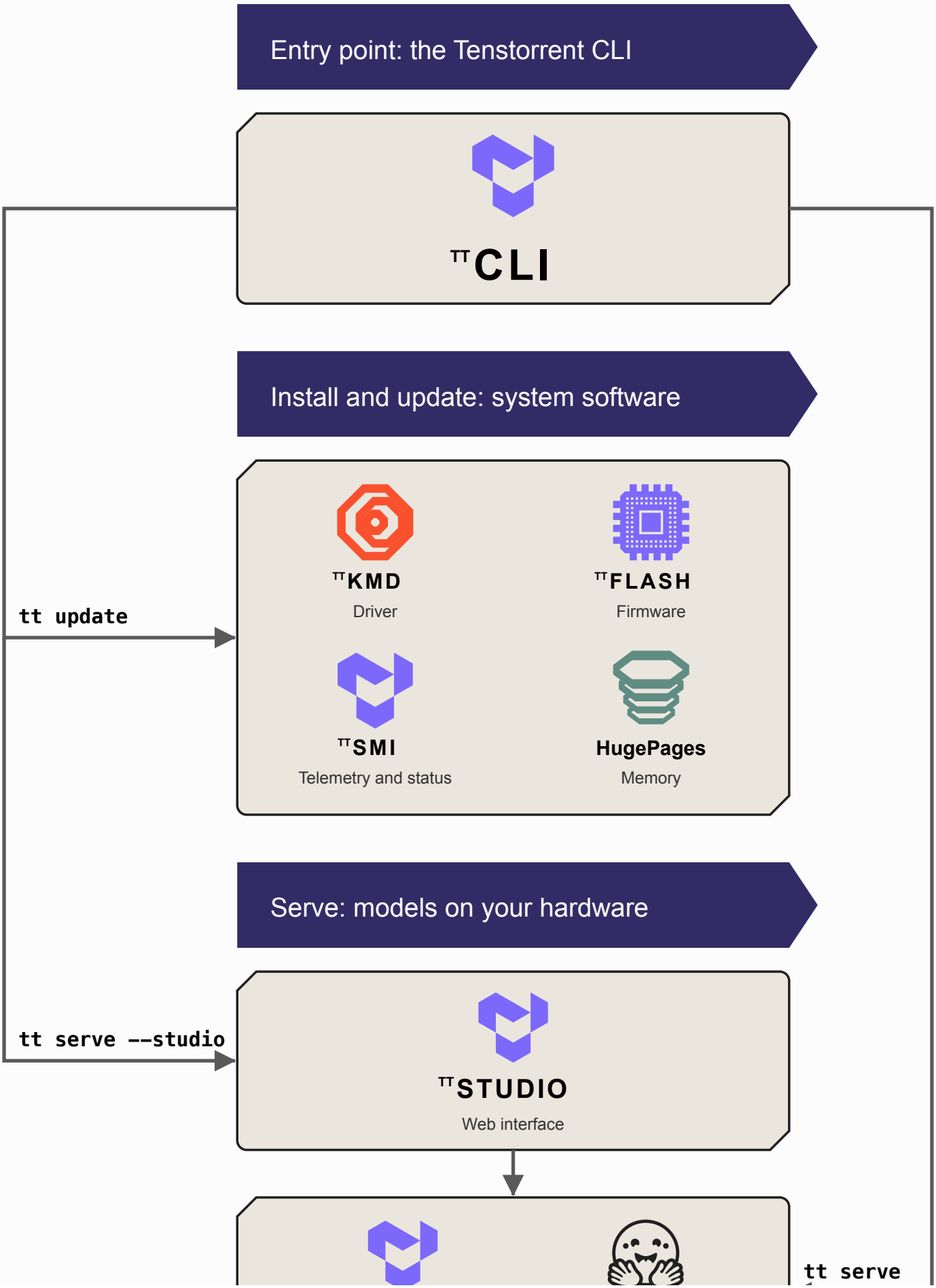
For real-time discussions, community Q&A, and to share your latest projects, we invite you to join our Discord channel. It's a great place to engage directly with other developers and members of the Tenstorrent community.

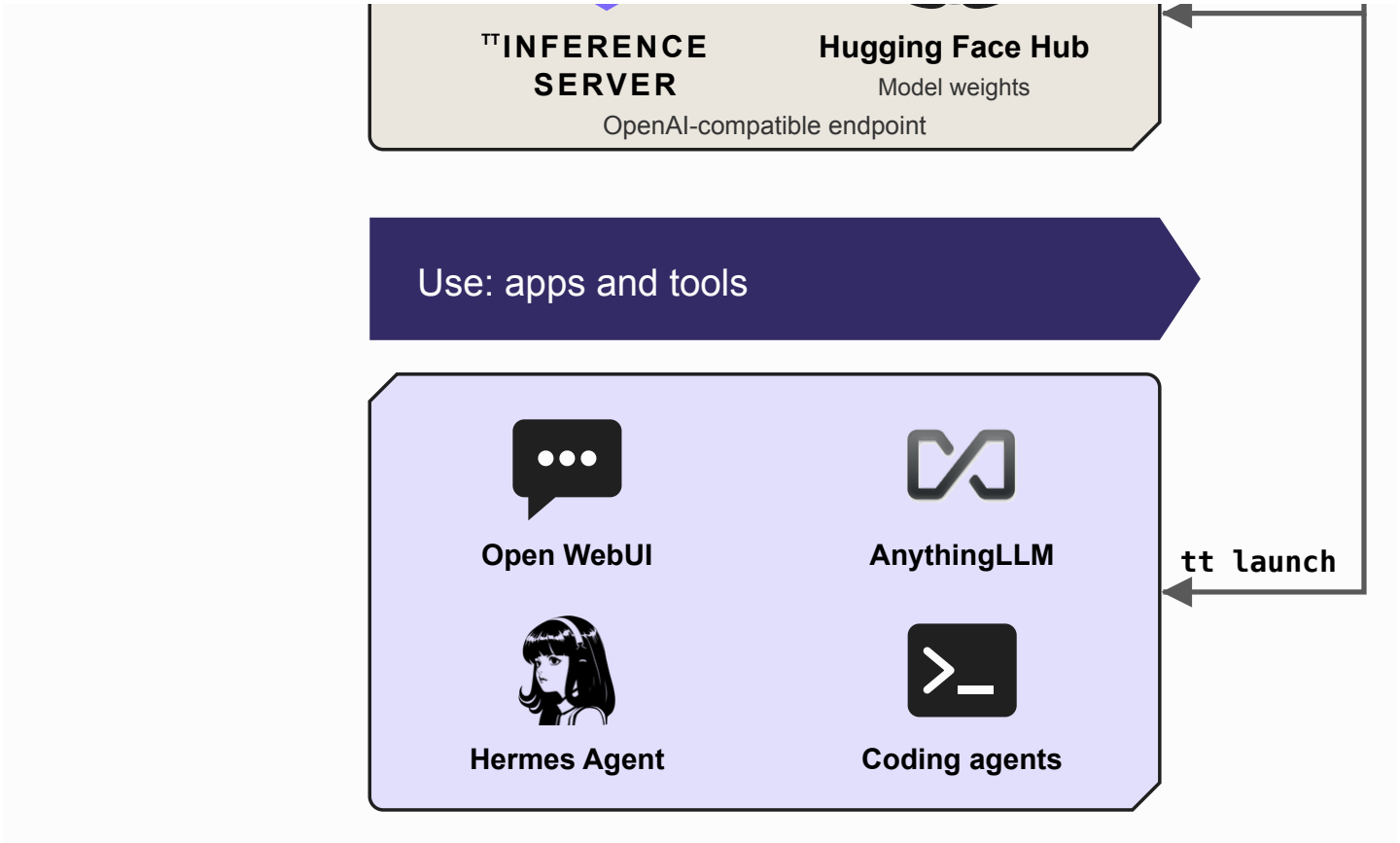
As always, if you have any questions, please raise a support request with our team. We're here to help, and are excited to see what you build.

PREVIOUS NEXT

Operating Tenstorrent Software

This document provides an overview of the main components you use to operate Tenstorrent hardware: the Tenstorrent CLI, `tt`, and TT-Studio. You will learn what each one is for, how they fit together with the software they manage, and where to find their source code. If you want to build on the stack instead, by compiling models or writing kernels, start with Building with Tenstorrent Software.


Entry point: the Tenstorrent CLI
TT CLI
Install and update: system software
TT KMD
Driver
TT FLASH
Firmware
tt update
TT SMI
Telemetry and status
HugePages
Memory
Serve: models on your hardware
tt serve --studio
TT STUDIO
Web interface
tt serve



tt: The Tenstorrent CLI

`tt` is the Tenstorrent command-line interface and the primary interface for operating your Tenstorrent hardware. It either implements a function itself or drives an existing Tenstorrent tool, such as TT-SMI or tt-installer, behind one stable set of commands.

Interact with `tt` when you want to set up a Tenstorrent system and run models on it without working through the individual tools yourself. It installs and updates the system software, checks that your devices are healthy, lists the models validated for your hardware, serves one as an OpenAI-compatible endpoint, and connects applications to it. Because it is a command-line tool, every step can be scripted and automated.

The main project is located in the tt-cli GitHub repository. Installing Tenstorrent Software walks through a first install with it.

TT-Studio: The Web Interface

TT-Studio is a web application for running and using AI models on Tenstorrent hardware from a browser. It wraps the same serving engine that `tt serve` uses behind a point-and-click interface, so you choose a model from a list filtered to your hardware, deploy it, and use it without touching a terminal.

Interact with TT-Studio when you want to try a model from a browser, or hand a running system to people who do not use a command line. It provides an interface for each kind of model: chat with document knowledge bases for language models, image and video generation for diffusion models, speech-to-text for recognition models, object detection for vision models, and a voice agent that chains them together. Every model TT-Studio runs is also served on an API endpoint, so applications can use it at the same time as the browser does. `tt serve <model> --studio` starts TT-Studio and deploys a model in one step; it also runs on its own, and can point at endpoints on Tenstorrent hardware elsewhere.

The main project is located in the tt-studio GitHub repository.

What tt Manages

System software

`tt update` installs and updates the system software: the kernel-mode driver (TT-KMD) that exposes the devices to Linux, the device firmware and the TT-Flash tool that writes it, the HugePages memory configuration the devices need, TT-SMI for telemetry and status, and a container runtime, normally Docker, for serving models. Tenstorrent tests these components together, and `tt update` brings a system onto the latest tested combination in one command, using tt-installer for the parts that install as root. One piece of system configuration sits outside it: on a multi-card Wormhole system the chips are linked into a mesh by `tt-topology`, a one-time step described in Installing Tenstorrent Software. Blackhole systems need no equivalent. `tt device` and `tt smi` then show which devices are present, along with their firmware, temperature, power, and clocks.

Model serving

`tt serve` serves a model as an OpenAI-compatible endpoint. For the models in Tenstorrent's tested catalog it runs TT-Inference-Server, which packages each model with the right runtime in a container and downloads its weights from the Hugging Face Hub. Community bundles published on the Hub run through tt-model-manager, since each bundle carries its own serving stack. `tt model list` shows the catalog for your hardware, filtered to the configuration `tt` detects, and it is not limited to large language models: it includes diffusion models for image and video generation, speech and audio models, and vision models. It is the authoritative list of models validated for your hardware; Tenstorrent does not test or maintain community bundles.

Apps and tools

`tt launch` connects a ready-made client to the model being served. Web UIs such as Open WebUI and AnythingLLM are pulled and run for you in a container; terminal coding agents such as OpenCode, Aider, pi, Qwen Code, and Hermes Agent are ones you install yourself, and `tt` configures them to reach your model. Because the endpoint speaks the OpenAI API, any other OpenAI-compatible client works too.

Choosing Between tt and TT-Studio

Both paths serve the same catalog, and a model served by one is reachable from the other, so the choice is about how you want to work:

- **Use** `tt` when you want an API endpoint for applications, scripts, or agents, when you are setting up or maintaining a system, or when you want to automate any of that.
- **Use TT-Studio** when you want to try models from a browser, when the people using the system are not terminal users, or when you want the built-in chat, generation, and recognition interfaces without connecting a separate client.

For monitoring your hardware while models run, `tt smi` gives a status snapshot and TT-Toplike reads power, temperature, and core activity in real time from the chips.

Where to Go Next

- **Set up a system** with Installing Tenstorrent Software: install `tt`, update the system software, and verify your hardware.
- **Serve more kinds of models** with the Serve Models guide, which walks the catalog, `tt serve`, and TT-Studio with a diffusion model as the example.
- **Deploy LLMs in production** with TT-Inference-Server directly.
- **Build on the stack** by reading Building with Tenstorrent Software, which introduces TT-Forge™, TT-NN™, and TT-Metalium™.

As always, if you have any questions, please raise a support request with our team.

PREVIOUS NEXT

Tenstorrent Documentation

No hardware yet? Get access to Tenstorrent silicon right in your browser with Tenstorrent Cloud Console

GET STARTED

Installing Tenstorrent Software

Set up drivers, firmware, and the TT-Metalium environment on your Tenstorrent hardware, then verify your install.

TT-VSCODE-TOOLKIT

Learn by Building

Get hands-on with a guided learning path — from making your first inference to building full applications with LLMs.

Popular Destinations

TT-Studio

Point-and-click web UI for deploying and interacting with models.

TT-Inference-Server

Deploy and serve models — the canonical source for validated model support.

TT-Metalium

Low-level C++ SDK for writing custom kernels on Tensix cores.

Get Started →

Installing Tenstorrent Software

Install drivers, firmware, and the TT-Metalium environment, then verify your setup.

Building with Tenstorrent Software

How TT-Forge, TT-NN, and TT-Metalium fit together — and which to use when.

Operating Tenstorrent Software

How the tt CLI and TT-Studio install, serve, and use models on your hardware — and when to use each.

Serve Models

Serve LLMs, diffusion, speech, and vision models with the tt CLI or TT-Studio, and get an OpenAI-compatible endpoint.

Manual Installation

Manually install system dependencies, drivers, and the software stack step by step.

Learn by Building

Get hands-on with guided, end-to-end lessons in the TT-VSCode-Toolkit.

Software →

TT-Forge™

End-to-end MLIR compiler. Compile models from PyTorch, JAX, and ONNX.

TT-XLA

PJRT-based frontend for running JAX and OpenXLA models on Tenstorrent hardware.

TT-NN™

High-level Python API of pre-optimized neural-network operations.

TT-Lang™

Python DSL for authoring fused custom operations.

TT-MLIR™

MLIR-based backend compiler infrastructure that lowers models onto Tenstorrent hardware.

TT-Metalium™

Low-level C++ SDK for writing custom kernels on Tensix cores.

Cloud-Native Support

Run Tenstorrent accelerators on Kubernetes — install, configure, and operate the device stack.

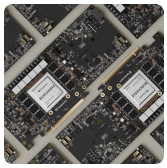
Hardware →

BLACKHOLE



TT-QuietBox® 2 (Blackhole)

Next-generation liquid-cooled Blackhole workstation.



Blackhole® PCIe Cards

Infinitely scalable, high-performance AI processing.



TT-QuietBox® (Blackhole)

Dead-quiet, liquid-cooled desktop AI.



Tenstorrent Galaxy® (Blackhole)

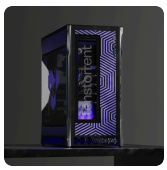
Scalable, enterprise-class AI server.

WORMHOLE



Wormhole™ PCIe Cards

Flexible, scalable processors built with Tensix Cores™.



TT-QuietBox® (Wormhole)

Dead-quiet desktop AI.



TT-LoudBox™ (Wormhole)

High-throughput workstation AI.



Tenstorrent Galaxy® (Wormhole)

Scalable, enterprise-class AI server.

Learn End-to-End through TT-VSCode-Toolkit →

Your First Inference

Set up the toolkit, detect hardware, and run your first model end-to-end.

Serving & APIs

Production inference with tt-inference-server and vLLM, plus image and video generation.

Applications

Real-world apps: coding assistants, AI agents, and video generation on QuietBox 2.

Custom Training

Datasets, fine-tuning, multi-device training, and training from scratch.

Cookbook

Hands-on projects: Game of Life, audio processing, fractals, and particle sims.

Compilers & Frameworks

Image classification with TT-Forge, JAX and PyTorch/XLA, and an intro to TT-Lang.

CS Fundamentals

RISC-V architecture, memory hierarchy, parallelism, synchronization, and complexity.

Advanced

Model bring-up bounties and exploring TT-Metalium at the kernel level.

Deployment

Deploy the toolkit and your work to the cloud with Koyeb.

Tools →

TT-Inference-Server

Deploy and serve models — the canonical source for validated model support.

TT-Studio

Point-and-click web UI for deploying a model — the fastest path for new users.

TT-SMI

Command-line device telemetry and firmware info — ships with the software stack.

TT-Toplike

Real-time terminal monitor: power, temperature, current, and core activity.

TT-NN Visualizer

Inspect model execution: graphs, memory plots, tensor and buffer views.

TT-Topology

Configure Ethernet routing on multi-card QuietBox and LoudBox systems.

TT-VSCode-Toolkit

Guided VS Code learning environment — browse tracks in the Learn section above.

TT-Blacksmith

Optimized ML training recipes for Tenstorrent hardware.

Explore the Ecosystem →

TT-Awesome

A curated directory of 115+ community and official projects, models, kernels, and research across 12 categories.

🪐 Planet Tenstorrent

The latest across the ecosystem — releases, papers, talks, videos, and demos, aggregated in one feed.

Tenstorrent on GitHub

All public source repositories — the software stack, compilers, tools, and demos.

Resources →



Request Support

Submit a support ticket and our team will help.



FAQ

Learn more about our hardware and software.



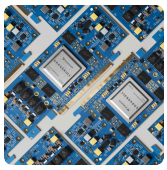
Contact Us

Get in touch with the Tenstorrent team.

Join Our Discord

Join the Tenstorrent developer community.

Legacy



Grayskull™

Tenstorrent's first-generation Tensix processor.



T7000

Earlier-generation Tenstorrent system.



T1000

Earlier-generation Tenstorrent system.

Installing Tenstorrent Software

This guide takes you from a fresh Tenstorrent system to one that is ready to run models. You will install the Tenstorrent CLI, `tt`, use it to update your system to the latest stable Tenstorrent software, and confirm your hardware is visible.

`tt` is the primary interface for operating your Tenstorrent hardware. If you want to develop against Tenstorrent's open-source repositories instead, start with TT-Metalium™, TT-NN™, or TT-Forge™.

Before You Begin

Note

Setting up your new TT-QuietBox® 2? Follow the Setup Guide.

Before you begin, ensure your host machine meets the following prerequisites:

- The host machine has an internet connection. `tt` downloads packages from PyPI and GitHub, the Tenstorrent package repositories, and model weights from the Hugging Face Hub.
- The host machine has a supported operating system installed (see below).
- You have administrator (`sudo`) privileges on the host machine. The driver, firmware tooling, and HugePages configuration are installed as root.
- The system is physically unboxed and set up according to your product's installation manual and safety guidelines.

Supported Operating Systems

Tenstorrent recommends Ubuntu LTS for all Tenstorrent software.

Operating system	Status
Ubuntu 22.04 LTS, Ubuntu 24.04 LTS	Supported
Other Linux distributions	Untested and considered experimental
macOS, Windows, WSL	Not supported. There is no driver, so nothing hardware-related work

BIOS Requirements

The BIOS for your host motherboard should configure the PCIe AER Reporting Mechanism to be set to `OS First`. Tenstorrent's TT-SMI software requires this setting to function properly.

Important

If you are using a TT-QuietBox, you do not have to worry about setting this option. It is set to `OS First` by default.

If you update or reset your BIOS for any reason, you must reconfigure the PCIe AER Reporting Mechanism setting to `OS First` to ensure TT-SMI functions correctly. This setting is typically located in the **Motherboard Information** section of your BIOS.

Install the Tenstorrent CLI Dependencies

The Tenstorrent CLI, `tt`, is a Python application. It needs Python 3.10 or newer with the `venv` module, plus `git` and `curl`. Install them with your system package manager. On Ubuntu:

```
sudo apt update && sudo apt install -y python3 python3-venv python3-pip git curl ca-certificates
```

`tt` must live in its own isolated Python environment so that it can safely update itself later. Tenstorrent recommends uv to manage that environment. Follow the setup instructions at https://docs.astral.sh/uv/ to install `uv`, then open a new terminal or source your shell configuration file (`source ~/.bashrc` for `bash`) so that `uv` is on your `PATH`.

Note

If you already use pipx, it works too. Install `tt` with `pipx install tenstorrent` in the next step instead of the `uv` command. Other installation options are described in the tt-cli developer notes.

Step 1: Install the Tenstorrent CLI

Note

If you're starting with a fresh TT-QuietBox® 2, we automatically install the Tenstorrent CLI for you. To check, run `tt --version` in your terminal. If you get a response, move to Step 2 below.

Install `tt` as a `uv` tool and make sure it is on your `PATH`:

```
uv tool install tenstorrent
uv tool update-shell
```

Open a new terminal or source your shell configuration file again, then confirm `tt` runs:

```
tt --version
```

The first time you run `tt` in a terminal it asks once whether you want to share anonymous usage telemetry. Please consider saying yes. Telemetry tells Tenstorrent which commands people use, how long they take, and which ones fail, and that is what guides what gets fixed and built next. It is strictly opt-in and records no prompts, file paths, hostnames, usernames, or IP addresses. You can see exactly what would be sent before you decide: set `TT_TELEMETRY_LOG_FILE=/tmp/tt-events.jsonl` on any command and read the file, which works without opting in and uploads nothing. You can change your answer at any time with `tt config set telemetry.enabled true` or `false`. The telemetry policy lists everything that is collected.

Run `tt` to see the command groups. The ones used in this guide are `update` and `device`; `model`, `serve`, and `launch` take over once the stack is installed.

Step 2: Install the Software Stack with `tt update`

Tenstorrent's system software stack consists of several components that are regularly tested together to ensure stability: device firmware, the kernel-mode driver (TT-KMD), HugePages configuration, TT-SMI, TT-Flash, and a container runtime for serving models. `tt update` brings your system up to the latest stable, tested combination of those components in one command. On a fresh machine it installs everything. On a machine that already has Tenstorrent software it changes only what is out of date, so you can rerun it at any time.

```
tt update
```

`tt update` installs the Tenstorrent tools into isolated environments under `~/.local/share/tenstorrent/`, where they cannot conflict with your own Python environment, then runs the matching tt-installer release non-interactively to install the driver, firmware, HugePages configuration, and Docker if no container runtime is present.

Required

Using sudo is required, so you must enter your user's password when prompted.

`tt update` also asks you to confirm before it flashes firmware, because flashing resets the devices. Stop any running model workloads first, then answer **y**.

On a TT-QuietBox 2 with a fast connection the whole update takes a few minutes.

Bring `tt` itself up to date

The installer that `tt update` runs also installs the `tt` release that was tested with that set of components, which can be older than the one you installed in Step 1. This page documents the latest release, so before you move on, put `tt` back on it:

```
tt self update
```

Keep both halves current over time: `tt update` for the system software and tools, and `tt self update` for `tt` itself. `tt` checks for a newer release once a day and prints a notice when one is available.

Reboot after the first install

`tt update` never reboots your machine for you. If this is the first time the driver was installed on this machine, or HugePages configuration changed, **reboot now** before continuing:

```
sudo reboot
```

If `tt update` installed Docker, it also adds your user to the `docker` group. That takes effect when you log in again, which the reboot takes care of.

Step 3: Verify Your Hardware

After rebooting, confirm the driver is loaded and that every Tenstorrent device in your system is visible:

```
tt device status
```

You should see one row per chip, with the board type, PCIe bus address, and live telemetry. The **Board** column tells you which generation you have: `p100`, `p150`, and `p300` boards are Blackhole; `n150` and `n300` boards are Wormhole. A few later steps apply to one generation only and say so. Here is an example from a TT-QuietBox 2 with four Blackhole p300 cards:

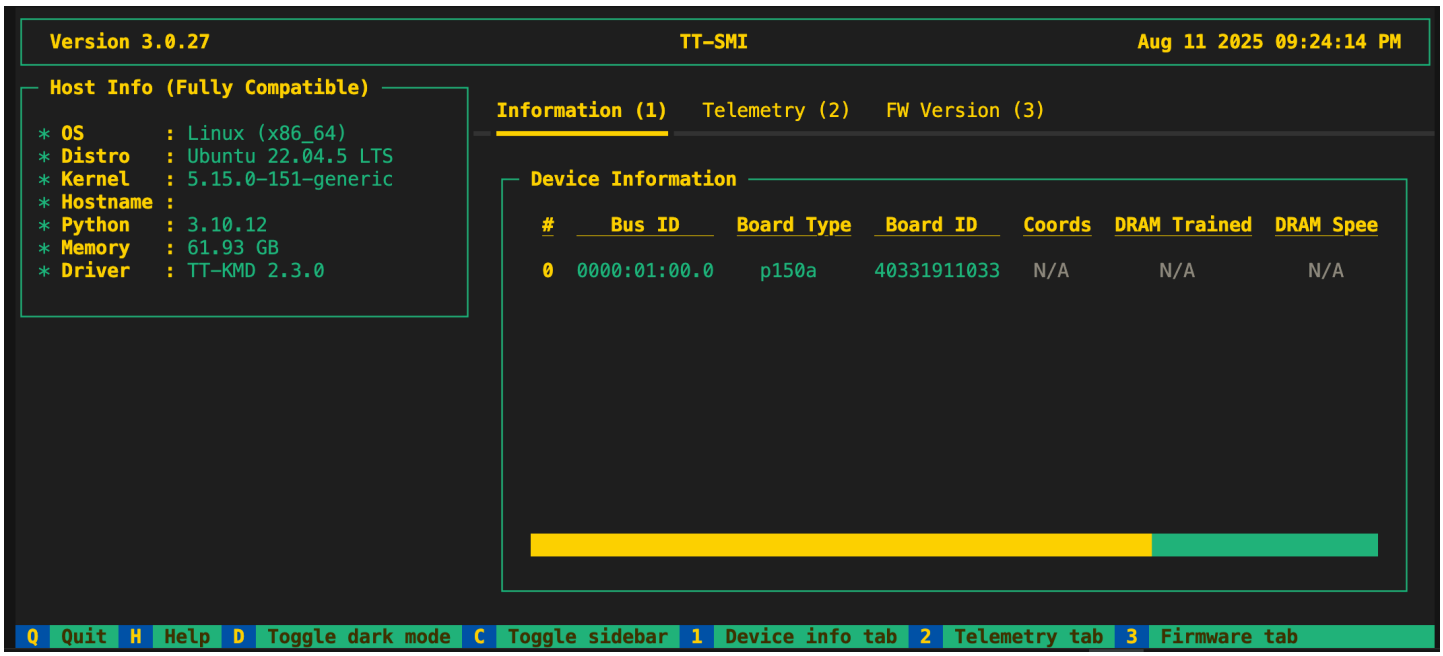
```
                Tenstorrent devices (driver: TT-KMD 2.11.0)
┏━━━┳━━━━━━━┳━━━━━━━━━━━━━┳━━━━━━━━┳━━━━━━━━┳━━━━━━━━━┳━━━━━━━━━┳━━━━━━┓
┃ # ┃ Board ┃ Bus ID      ┃ Temp   ┃ Power  ┃ AICLK   ┃ Voltage ┃ DRAM ┃
┡━━━╇━━━━━━━╇━━━━━━━━━━━━━╇━━━━━━━━╇━━━━━━━━╇━━━━━━━━━╇━━━━━━━━━╇━━━━━━┩
│ 0 │ p300c │ 0000:01:00.0 │ 43.9 °C │ 31.0 W │ 800.0 MHz │ 0.71 V  │ True │
│ 1 │ p300c │ 0000:02:00.0 │ 44.4 °C │ 17.0 W │ 800.0 MHz │ 0.71 V  │ True │
│ 2 │ p300c │ 0000:03:00.0 │ 43.6 °C │ 16.0 W │ 800.0 MHz │ 0.72 V  │ True │
│ 3 │ p300c │ 0000:04:00.0 │ 45.1 °C │ 16.0 W │ 800.0 MHz │ 0.72 V  │ True │
└───┴───────┴──────────────┴────────┴────────┴─────────┴─────────┴──────┘
```

A Wormhole n300 card carries two chips and shows two rows, `n300 L` with the card's PCIe bus address and `n300 R` with `N/A`, because the second chip is reached through the first. A TT-LoudBox or Wormhole TT-QuietBox with four n300 cards therefore shows eight rows:

```
                Tenstorrent devices (driver: TT-KMD 2.11.0)
┏━━━┳━━━━━━━┳━━━━━━━━━━━━━┳━━━━━━━━┳━━━━━━━━┳━━━━━━━━━━┳━━━━━━━━━┳━━━━━━━┓
┃ # ┃ Board ┃ Bus ID      ┃ Temp   ┃ Power  ┃ AICLK    ┃ Voltage ┃ DRAM  ┃
┡━━━╇━━━━━━━╇━━━━━━━━━━━━━╇━━━━━━━━╇━━━━━━━━╇━━━━━━━━━━╇━━━━━━━━━╇━━━━━━━┩
│ 0 │ n300 L │ 0000:31:00.0 │ 45.9 °C │ 28.0 W │ 1000.0 MHz │ 0.89 V  │ True  │
│ 1 │ n300 L │ 0000:4b:00.0 │ 44.8 °C │ 27.0 W │ 1000.0 MHz │ 0.88 V  │ True  │
│ 2 │ n300 L │ 0000:b1:00.0 │ 44.4 °C │ 29.0 W │ 1000.0 MHz │ 0.9 V   │ True  │
│ 3 │ n300 L │ 0000:ca:00.0 │ 43.1 °C │ 29.0 W │ 1000.0 MHz │ 0.9 V   │ True  │
│ 4 │ n300 R │ N/A          │ 37.5 °C │ 24.0 W │ 1000.0 MHz │ 0.87 V  │ True  │
│ 5 │ n300 R │ N/A          │ 36.4 °C │ 20.0 W │ 1000.0 MHz │ 0.91 V  │ True  │
│ 6 │ n300 R │ N/A          │ 37.4 °C │ 24.0 W │ 1000.0 MHz │ 0.91 V  │ True  │
│ 7 │ n300 R │ N/A          │ 36.1 °C │ 23.0 W │ 1000.0 MHz │ 0.9 V   │ True  │
└───┴────────┴──────────────┴─────────┴────────┴────────────┴─────────┴───────┘
```

The number of rows should match the number of Tenstorrent chips installed in your system: one per Blackhole card, two per n300 card. For more detail on a device, such as its firmware versions and board ID, run `tt device info`. For a live, interactive view, run `tt smi`, which hands your terminal to the TT-SMI interface. Press **Q** to leave it.

Here is an example of the TT-SMI interface for a system containing a single p150a device:



Important

If the number of listed devices does not match what you expect, see Troubleshooting below. If the problem persists, contact support and we will assist you.

Congratulations, your system software is installed and your hardware is ready to use.

Step 4 (Wormhole only): Configure the Mesh Topology

Skip this step on Blackhole systems and on single-card Wormhole systems.

A Wormhole TT-LoudBox or TT-QuietBox connects its n300 cards to each other over Ethernet, and the models in the catalog expect those links to form a mesh. `tt update` installs the driver and firmware but does not configure the mesh, and a system can arrive, or come back from a firmware update, with the chips **Isolated**. Check first:

```
tt-topology -ls
```

If it reports `Mesh`, continue to the next page. If it reports `Isolated`, or the command is not found, run the script below once. It installs TT-Topology into a temporary environment, resets every chip, and writes the mesh configuration. It needs no `sudo`, takes about two minutes, and ends with "Completed reset on 8 chips" on a four-card system.

```
TMP_DIR=$(mktemp -d); (trap 'echo "---"; echo "Cleaning up..."; if type deactivate &>/dev/null; then deactivate; fi; echo "Removing temporary directory: $TMP_DIR"; rm -rf "$TMP_DIR"; cd; echo "Cleanup complete."' EXIT; trap 'echo -e "\033[0;31m!!! ERROR: Failed to configure mesh topology\033[0m"' ERR; set -e; cd "$TMP_DIR"; echo "Working in temporary directory: $TMP_DIR"; echo "---"; echo "Creating Python virtual environment..."; python3 -m venv tt-topology-venv; source tt-topology-venv/bin/activate; echo "Virtual environment activated."; echo "---"; echo "Installing tt-topology from git..."; pip install --quiet git+https://github.com/tenstorrent/tt-topology.git; echo "tt-topology installed."; echo "---"; echo "Running tt-topology command. This may take a moment..."; tt-topology -l mesh; echo "---"; echo "Script finished successfully.";)
```

Run `tt-topology -ls` again and confirm `Mesh`. Serving a model without this step stops at system validation with `ModelSpec requires SystemTopology.MESH, detected ISOLATED`.

Troubleshooting

`tt device status` reports no devices

If `tt device status` exits with a message that the driver may not be installed or no card is seated:

1. If you have not rebooted since `tt update` installed the driver, reboot now.
2. Run `lspci -d 1e52:` to check whether the card has enumerated on the PCIe bus. If it returns nothing, the host cannot see the card at all. Each Blackhole card and each n150 card appears once; an n300 card also appears once, because its second chip has no PCIe address of its own.
3. For Blackhole PCIe products such as p100 and p150, verify the card is being supplied power by checking that the fan spins and the green power LED illuminates when the system boots. If the light is on, try resetting the card with `tt device reset`.
4. Confirm the BIOS setting described in BIOS Requirements.

Docker commands are refused with a permission error

`tt` talks to Docker as your user, never through `sudo`. If `tt update` just installed Docker, log out and back in (or reboot) so that your membership in the `docker` group takes effect. If Docker was already installed before you ran `tt update`, the installer leaves it as it found it and does not add you to the group. Add yourself, then log out and back in:

```
sudo usermod -aG docker $USER
```

You can check with `docker run hello-world`.

Getting help

`tt report bundle` collects a redacted support bundle with your environment details, a TT-SMI snapshot, and recent logs, and opens a prefilled email to Tenstorrent support with it attached. `tt report issue` opens a prefilled GitHub issue on the tt-cli repository.

Alternative Installation Methods

`tt update` is the recommended way to install the system software, and it runs tt-installer for you with the tested versions. If you prefer to drive tt-installer interactively yourself and choose each of its options, you can run it directly:

```
/bin/bash -c "$(curl -fsSL https://tenstorrent.ai/install.sh)"
```

For advanced users or developers who prefer to install and configure each component by hand, refer to the manual software installation guide.

What's Next

With the software stack installed, `tt` takes you the rest of the way to a running model. `tt model list` shows the models validated for the hardware it detected, and `tt model info <model>` reports the disk space, host memory, and validation status of each one. Catalog models download their weights from the Hugging Face Hub, so log in once with `tt model login`; gated models such as Llama and Gemma also require accepting the license on the model's Hub page first. `tt serve <model>` then installs TT-Inference-Server, downloads the weights, and starts an OpenAI-compatible endpoint on port 20000, and `tt model curl "<prompt>"` sends it a first request. The first serve of a model downloads and compiles a lot, so expect it to take a while; later serves are much faster.

An endpoint is the foundation; `tt launch` puts an application on top of it. `tt launch openwebui` starts a chat interface in your browser, and `tt launch list` shows the other web UIs and terminal coding agents available on your machine. If you prefer a point-and-click experience, `tt serve <model> --studio` starts the model inside TT-Studio, a web application that serves models and gives you a browser interface for them in one place. The Serve Models guide walks through all of this in detail. Run `tt` at any time to see the command groups, and `tt <command> --help` for the options of each.

Where to Go Next

You now have the full stack installed. From here, pick the part of the stack that matches what you want to build:

- **Serve more kinds of models** by following the Serve Models guide: choosing a model for your hardware, serving LLMs, diffusion, speech, and vision models from the CLI, and running the same models through TT-Studio's web interface.
- **Deploy LLMs in production** with TT-Inference-Server, the serving engine behind `tt serve`. Its repository is the authoritative source for which models are validated on your hardware.
- **Monitor your hardware** with `tt smi` for a status snapshot, or TT-Toplike for a live view of power, temperature, and core activity while a model serves requests.
- **Use a web interface** with TT-Studio, started through `tt serve <model> --studio`.
- **Understand the software stack**, including how TT-Forge™, TT-NN™, and TT-Metalium™ relate to each other, in Building with Tenstorrent Software.
- **Compile your own PyTorch, JAX, or ONNX models** with TT-Forge™.
- **Run and build models in Python** with TT-NN™, or **write custom kernels** with TT-Metalium™. `tt update` also installed a `tt-metalium` command that opens a shell inside the TT-Metalium container, with your home directory mounted and TT-NN ready to import; the image, about 4.5 GB, is pulled the first time you run it. Start with the Metalium guide to learn about the architecture.
- **Learn by building** with the hands-on lessons in the TT VS Code Toolkit.

Need Additional Support?

If you encounter any issues, or have a question that isn't covered in the documentation, please raise a support request. Our team will review your request and provide assistance.